
LAPORAN PRAKTIKUM INTERNET OF THINGS



Disusun oleh :

Nama : Restu anggara

NPM : 25782051

Kelas : TRI-3B

Dosen :

Dr.Ir.Septafiansyah Dwi Putra,S.T.,M.T.

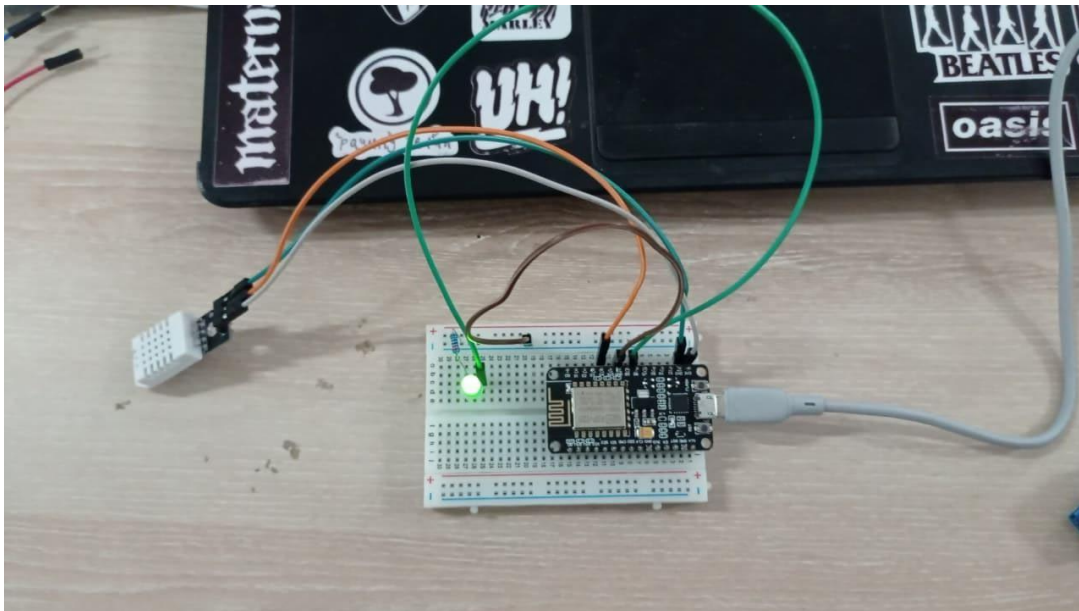
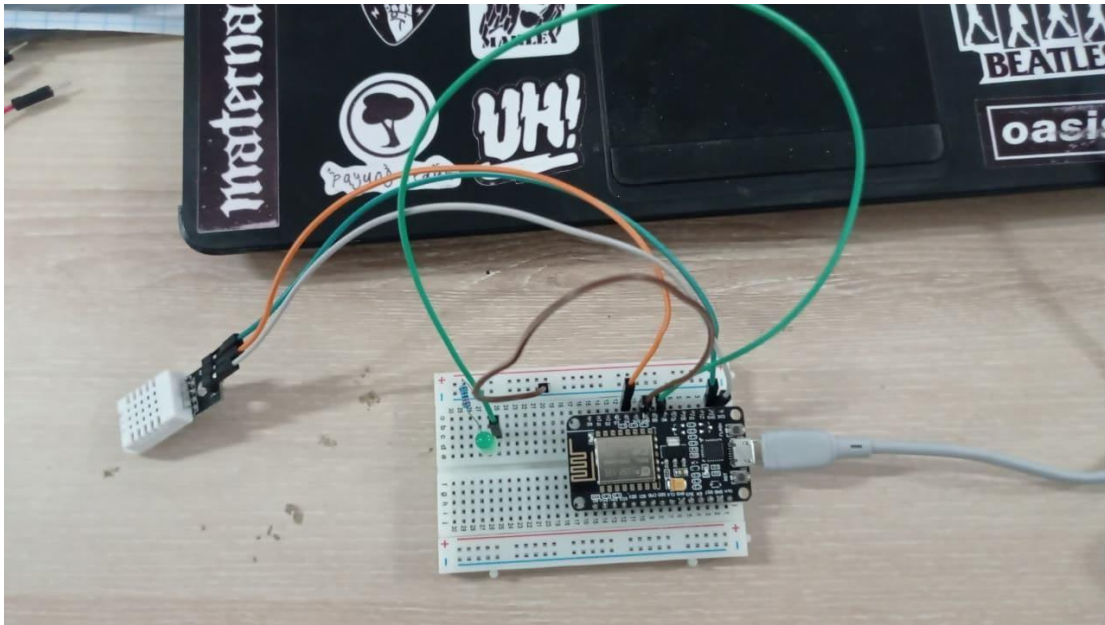
Agus Ambarwari, S.Pd.,M.Kom.

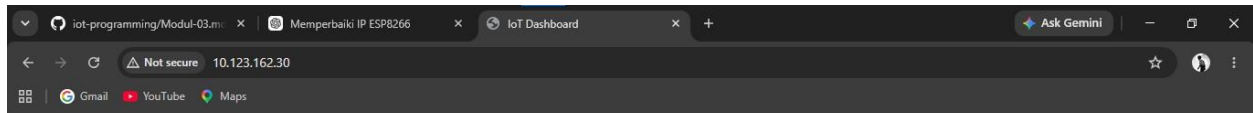
Andi Saptono,S.Kom

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG**

2026/2027

3.5.1 Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)





ESP8266 Web Server

Suhu Saat Ini: **1.30** Celcius

Kendali Relay

ON

OFF

```
09:10:03.259 -> .....
09:12:02.985 -> IP Address Server Anda:
09:12:02.985 -> 10.123.162.30
```

Penjelasan:

Praktikum ini bertujuan membuat NodeMCU ESP8266 sebagai web server yang dapat membaca suhu dari sensor DHT dan mengontrol relay melalui halaman web.

Sensor DHT dihubungkan ke D4 (GPIO 2) untuk membaca suhu, sedangkan relay dihubungkan ke D6 (GPIO 12) sebagai output. NodeMCU kemudian dihubungkan ke Wi-Fi menggunakan SSID dan password yang telah ditentukan.

Program menggunakan ESP8266WebServer pada port 80 untuk membuat web server. Nilai suhu dari sensor ditampilkan pada halaman web dengan mengganti placeholder %TEMPERATURE% menggunakan `html.replace()`.

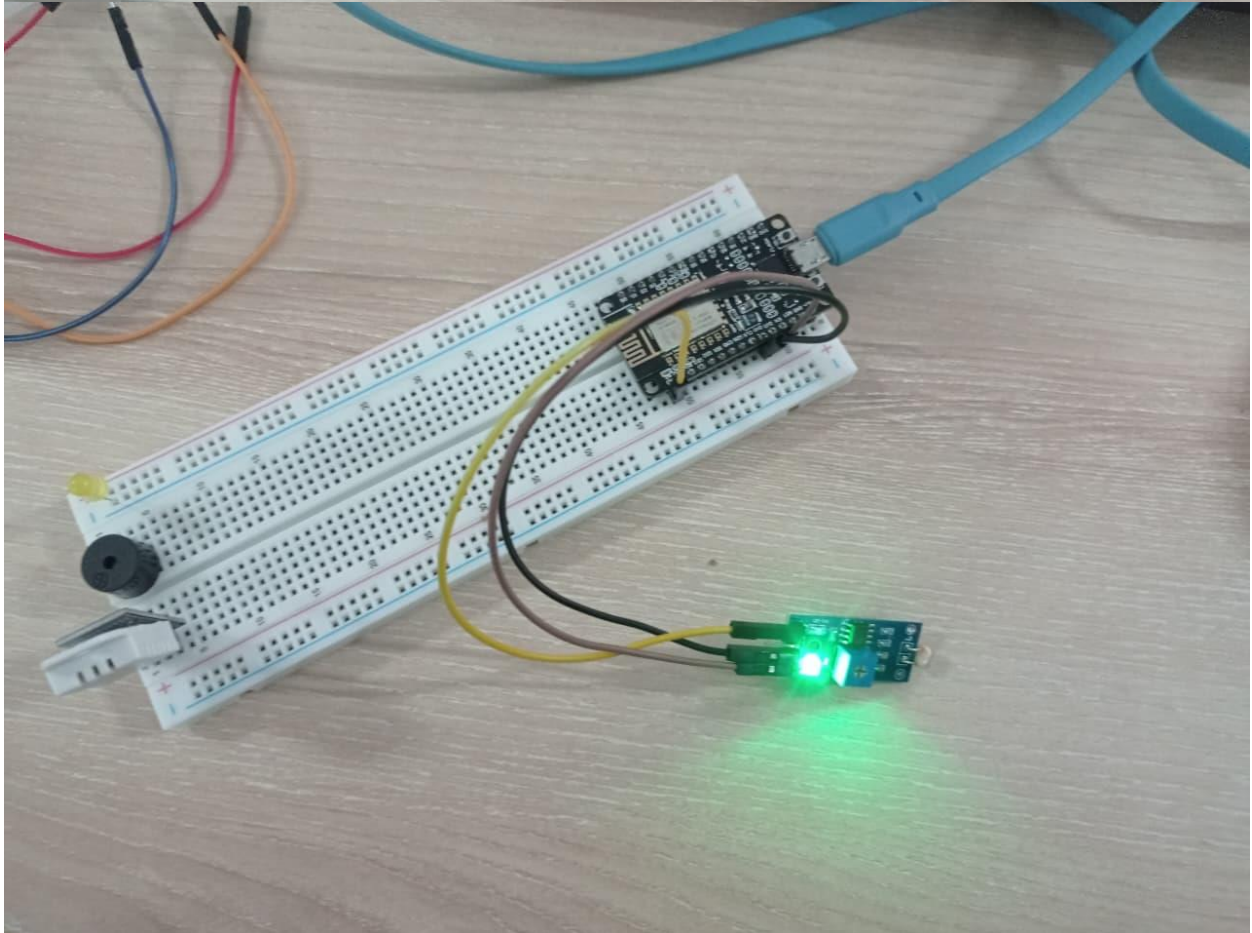
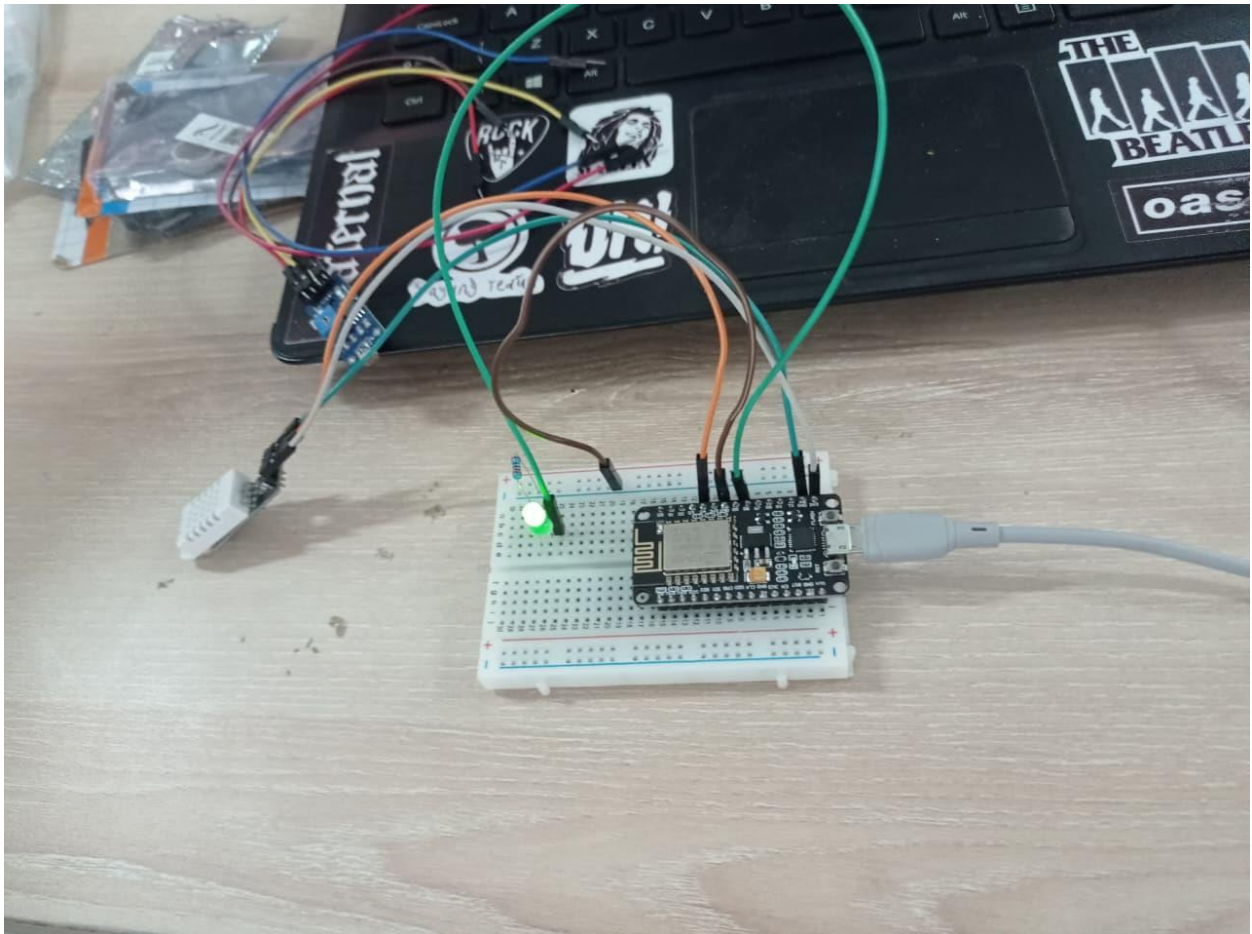
Terdapat dua tombol pada halaman web, yaitu ON dan OFF. Tombol tersebut menjalankan fungsi untuk mengatur kondisi relay. Setelah program berhasil dijalankan, alamat IP NodeMCU dapat dilihat melalui Serial Monitor dan dibuka melalui browser pada perangkat yang terhubung ke WiFi yang sama, ip yang saya dapatkan :

09:12:02.985 -> IP Address Server Anda:

09:12:02.985 -> 10.123.162.30

Hasilnya, suhu dapat ditampilkan pada halaman web dan relay dapat dikontrol melalui tombol ON dan OFF.

3.5.2 Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)



```
sketch_sep21a.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3  #include <DHT.h>
4
5  const char* ssid = "Aaauuu";
6  const char* password = "11111111";
7
8  ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT11);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18 | <meta name="viewport" content="width=device-width, initial-scale=1">
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

```
10:26:06.110 -> .....
10:27:48.149 -> IP Address Server Anda:
10:27:48.149 -> 10.59.206.30
11:19:13.812 -> ...
11:46:28.860 -> IP Address Server Anda:
11:46:28.860 -> 10.59.206.30
```

Code client:

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>

const char* ssid = "NAMA_WIFI_ANDA";
const char* password = "PASSWORD_WIFI_ANDA";

const char* serverName = "http://192.168.1.15/relay/on";
const byte ldrPin = A0;

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nClient Terhubung ke Wi-Fi!");
}

void loop() {
  int ldrValue = analogRead(ldrPin);

  if ((WiFi.status() == WL_CONNECTED) && (ldrValue < 300)) {
    WiFiClient client;
    HTTPClient http;

    http.begin(client, serverName);
    int httpResponseCode = http.GET();

    Serial.print("HTTP Response code: ");
    Serial.println(httpResponseCode);

    http.end();
    delay(10000);
  }
  delay(2000);
}
```

Penjelasan kode:

- WiFiClient client; dan HTTPClient http;; Kelas pemanggil untuk memulai eksekusi permintaan *client* jarak jauh (mirip fungsi aplikasi *browser* di laptop).
- http.begin(client, serverName);: Menargetkan sasaran URL *server*.
- int httpResponseCode = http.GET();: Mengeksekusi ketukan pintu (*Request* HTTP GET). Jika Server merespons sukses, nilai variabel ini akan berisi angka standar jaringan seperti 200 (OK) atau 303 (Redirect). Jika URL rusak atau *Server offline*, variabel akan mereturn nilai negatif.
- delay(10000);: *Cooldown* panjang 10 detik ini ditambahkan agar *Client* tidak memborbardir (*flood/DDoS*) Server dengan permintaan on secara terus-menerus setiap detik ketika kondisinya sedang gelap.

Penjelasan:

NodeMCU Server menggunakan **sensor DHT22 dan led**. DHT22 digunakan untuk membaca suhu, sedangkan led digunakan sebagai output yang dapat dikendalikan oleh Server. Server menerima permintaan HTTP dari Client untuk menyalakan relay.

NodeMCU Client menggunakan **sensor LDR** untuk membaca kondisi cahaya. Jika kondisi gelap, Client mengirimkan permintaan HTTP melalui Wi-Fi ke Server.

Alurnya:

LDR → Client → Wi-Fi → Server → led

DHT22 tetap berada di **Server** dan digunakan untuk menampilkan suhu pada halaman web.

3.6 Latihan

Kerjakan soal berikut secara mandiri untuk menguji penalaran dan modifikasi jaringan.

- **Latihan 1 (Pemahaman Jaringan):** Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

Jawaban:

Kami menggunakan mode jaringan *Station (STA)* karena kedua praktikum membutuhkan NodeMCU terhubung ke jaringan Wi-Fi yang sama dengan perangkat lain. Pada Praktikum 1, NodeMCU bertindak sebagai Web Server yang dapat diakses melalui jaringan Wi-Fi. Pada Praktikum 2, NodeMCU bertindak sebagai HTTP Client yang mengirim request ke Server. Jika menggunakan mode *Access Point (AP)*, NodeMCU akan membuat jaringan sendiri sehingga topologi komunikasi pada praktikum menjadi berbeda. Oleh karena itu, mode *STA* lebih sesuai karena NodeMCU dapat terhubung ke jaringan Wi-Fi yang sudah tersedia.

- **Latihan 2 (Modifikasi Antarmuka & Telemetri):** Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:
 - i. **Menampilkan Kelembapan:** Tambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">` (contoh: `<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>`), lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman *web*.
 - ii. **Tombol Dinamis Tunggal (*Toggle Button*):** Ubah antarmuka agar **hanya menggunakan 1 buah tombol tunggal** yang statusnya adaptif mengikuti kondisi LED saat ini:
 - Jika LED sedang **OFF**, tombol menampilkan tulisan "**NYALAKAN (ON)**" (misal: warna hijau/biru) dan mengarahkan aksi ke `/relay/on`.
 - Jika LED sedang **ON**, tombol otomatis berubah menjadi tulisan "**MATIKAN (OFF)**" (warna merah) dan mengarahkan aksi ke `/relay/off`. (Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti `%RELAY_BUTTON%`, lalu di fungsi `handleRoot()`, periksa status pin `relay/LED` menggunakan `digitalRead(relayPin)` sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai). Tuliskan potongan baris kode HTML dan fungsi `handleRoot()` yang Anda ubah!



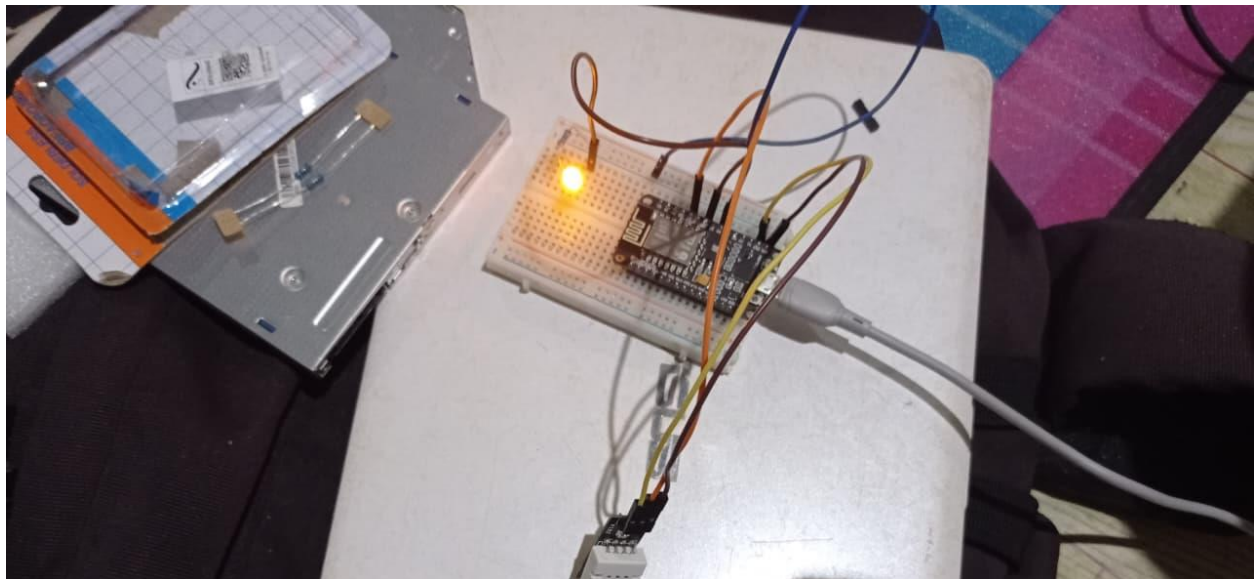
ESP8266 Web Server

Suhu Saat Ini: **2.70** Celcius

Kelembapan: **21.60** %

Kendali Relay

MATIKAN (OFF)



Kode yang ditambahkan:

```
float h = dht.readHumidity();
```

```
<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
```

```
%RELAY_BUTTON%
```

```
if (digitalRead(relayPin) == LOW) {  
  html.replace(  
    "%RELAY_BUTTON%",  
    "<a href=\"/relay/on\"><button class=\"btn-on\">NYALAKAN (ON)</button></a>"  
  );  
}
```

- **Latihan 3 (Analisis Kode Client):** Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

Jawaban :

Jika NodeMCU Server teman saya mendadak kehilangan daya atau offline, maka Client tidak akan mendapatkan HTTP Response Code seperti 200 atau 303. Pada Serial Monitor akan muncul nilai error negatif karena Client gagal terhubung ke alamat Server yang dituju. Hal ini terjadi karena `http.GET()` tidak mendapatkan respons dari Server. Jadi, hasil tersebut menunjukkan bahwa Server sedang offline atau tidak dapat diakses oleh Client.

- **Latihan 4 (Pengembangan Algoritma):** Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

Saya memodifikasi kondisi if agar Client hanya mengirim HTTP GET Request ketika suhu yang dibaca sensor DHT lebih dari 35°C.

```
if ((WiFi.status() == WL_CONNECTED) && (dht.readTemperature() > 35)) {
  // kirim HTTP GET Request
}
```

Jadi, jika suhu $\leq 35^{\circ}\text{C}$, Client tidak mengirim request. Jika suhu $> 35^{\circ}\text{C}$ dan Wi-Fi terhubung, Client akan mengirim request ke Server.

3.7 Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur

Antarmuka (*User Interface*) IoT tidak boleh statis. Saat ini, *Web Server* kita tidak mampu memperbarui informasi suhunya secara otomatis ke layar pengunjung tanpa diperintah. Kita harus menyelesaikannya.

Skenario Proyek (*Web Dashboard*): Anda akan bertindak sebagai *Full-Stack IoT Developer* untuk memutakhirkan kapabilitas *Web Server* pada Latihan 2, sembari mengevaluasi limitasi arsitektur tradisional HTTP.

- Tujuan: Mampu memanipulasi struktur kerangka HTML mentah yang tersimpan di memori NodeMCU untuk mencapai interaktivitas tampilan otomatis (*Auto-Refresh*).
- Tingkat Kesulitan: Menengah
- Instruksi:
 - i. Lakukan modifikasi kode hanya pada sisi NodeMCU Server (Latihan 2).
 - ii. Modifikasi variabel Raw Literal `index_html`. Sisipkan perintah *Meta Refresh HTML* (sebuah Tag standar web kuno namun ampuh) persis di dalam cakupan *tag* `<head>` untuk memaksa *browser* memuat ulang halaman secara sekuensial setiap 5 detik.
 - iii. *Hint (Petunjuk)*: Sintaks standar yang harus disisipkan adalah `<meta http-equiv="refresh" content="5">`.
 - iv. Setelah selesai, buka *browser*, dan amati layarnya. Biarkan selama 20 detik tanpa menyentuh *mouse/keyboard*.
- Instruksi Analisis Arsitektur:
 - i. Setelah Auto-Refresh berjalan, cobalah tekan tombol *ON / OFF* (kendali LED) dari *browser*.
 - ii. Apa kelemahan visual/interaksi (*flickering*) yang Anda alami saat menggunakan metode *Meta Refresh* HTTP konvensional seperti ini untuk sistem yang mengklaim diri sebagai *Real-Time Dashboard*? Tulis evaluasi logis Anda di Laporan Praktikum (Anda bisa membahas masalah *bandwidth*, layar berkedip putih, atau potensi bentrokan jika Anda mengklik tepat di detik ke-5).
- Kriteria Keberhasilan:
 - Nilai suhu dan kelembapan di *browser* otomatis berganti seiring kondisi lingkungan tanpa perlu mengklik tombol *refresh* di *browser*.
 - Jawaban analisis mengenai kelemahan protokol HTTP GET statis tertuang jelas di dokumentasi tugas.

Jawaban:



ESP8266 Web Server

Suhu Saat Ini: **31.20** Celcius

Kelembapan: **68.10** %

Kendali Relay



ESP8266 Web Server

Suhu Saat Ini: **31.30** Celcius

Kelembapan: **67.00** %

Kendali Relay



Kode modifikasi:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta http-equiv="refresh" content="5">
```

Penjelasannya:

- **meta** → tag HTML untuk memberikan informasi/perintah tertentu kepada browser.
- **http-equiv="refresh"** → memberi perintah kepada browser untuk melakukan refresh halaman.
- **content="5"** → menentukan waktu refresh, yaitu **setiap 5 detik**.

- Karena diletakkan di dalam **<head>**, browser akan menjalankan perintah tersebut saat halaman dibuka.

Jadi, setelah kode ini ditambahkan, browser akan otomatis meminta halaman NodeMCU Server kembali setiap 5 detik. Akibatnya, nilai **suhu dan kelembapan terbaru** akan ditampilkan tanpa pengguna menekan tombol refresh.

Kesimpulan:

Setelah menggunakan Meta Refresh setiap 5 detik, halaman browser dapat memperbarui nilai suhu dan kelembapan secara otomatis. Namun, metode ini memiliki kelemahan karena seluruh halaman web dimuat ulang setiap 5 detik. Hal tersebut dapat menyebabkan layar berkedip atau flickering dan menggunakan bandwidth lebih banyak karena browser harus meminta dan memuat kembali seluruh halaman. Selain itu, jika tombol ON/OFF ditekan tepat ketika proses refresh terjadi, dapat terjadi bentrokan atau interaksi terasa kurang responsif. Jadi, Meta Refresh cukup sederhana untuk membuat tampilan otomatis, tetapi kurang ideal untuk sistem yang membutuhkan tampilan real-time yang lebih halus.